

Contents

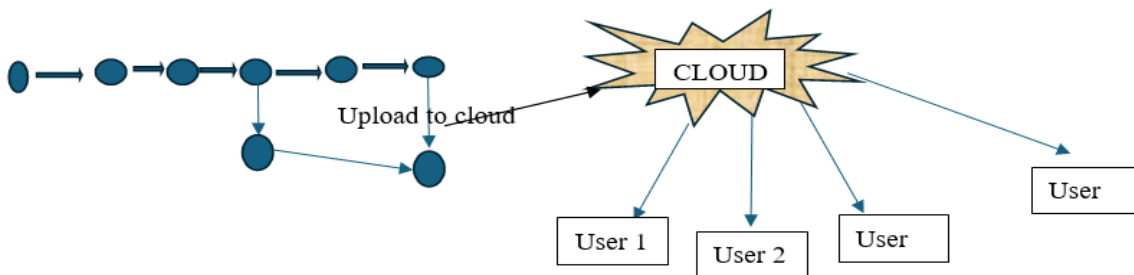
Git and GitHub.....	1
Working with Git and Bash commands - Local Repo	4
Working with Git Bash commands	4
Working with Git commands	5
Git workflow	6
Create remote Repo.....	7
How do I get my code on Github	7
Option 1	8
Option 2 - Git clone.....	9

Git and GitHub

Git and GitHub

Git	Github
Git is the version control software that runs locally on your machine. You don't need to register for an account. You don't need the internet to use it. You can use Git without ever touching Github	Github is a service that hosts Git repositories in the cloud and makes it easier to collaborate with other people. You do need to sign up for an account to use Github. It's an online place to share work that is done using Git.

From the previous example, all these are all on local machines. What if the local machine is lost or goes bad. All work is lost. I can upload my history to cloud and recover from the cloud. Using Github does the same job. The other advantage is that user/owner of work can share with others in the team as well.



Where Do We Start?

Below are the steps to set up Git and GitHub

- 1> Install Git
- 2> Create a local repository
- 3> Connect to the remote repository
- 4> Push the file to GitHub
- 5> Basic Git commands

1> Signing up for a new personal GitHub account

1. Navigate to <https://github.com/>.
2. Click **Sign up**.
3. Alternatively, click on **Continue with Google** to sign up using social login.
4. Follow the prompts to create your personal account.

During signing up, you'll be asked to verify your email address. Without a verified email address, you won't be able to complete some basic GitHub tasks, such as creating a repository.

Getting started: <https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github>

Tutorial: [How to Create a GitHub Account : https://www.youtube.com/watch?v=RnWDKWmaQ8s](https://www.youtube.com/watch?v=RnWDKWmaQ8s)

2> *Download and install: Git and Git Bash command line interface.*

Download and install: <https://git-scm.com/>

Tutorial to install Git: <https://www.youtube.com/watch?v=tRZGeaHPoaw>

Getting started: <https://docs.github.com/en/get-started/learning-to-code/getting-started-with-git>

Working with Git and GitHub: <https://www.youtube.com/watch?v=d3FkIA9RtIA>

OR

2> Download and install: GitHub Desktop Client

Instead of using Git from command line, GitHub Desktop Client allows us to use Git from graphical interface. If you have GitHub Desktop installed, you can use it to clone repositories and not deal with SSH keys.

Download and install: <https://github.com/apps/desktop>

Tutorial link: <https://www.youtube.com/watch?v=8Dd7KRpKeaE>

<https://www.youtube.com/watch?v=MaqVvXv6zrU>

Getting started: <https://docs.github.com/en/get-started/learning-to-code/getting-started-with-git#learning-git-basics-with-github-desktop>

Git Is (Primarily) A Terminal Tool Git was created as command-line tool. To use it, we run various git commands in a Unix shell. This is not the most user-friendly experience, but it's at the very core of Git!

How to get Git Bash in GitHub Desktop?

To get and use Git Bash inside GitHub Desktop, you need to **download Git for Windows separately** and then select it as your default shell inside the GitHub Desktop settings.

Follow this quick setup process to link them together:

Step 1: Install Git Bash on Your PC

1. Go to the official [Git for Windows Download Page](#).
2. Click **Download** to get the installer package.
3. Run the downloaded installer file (.exe).
4. Click **Next** through the setup prompt windows to accept the default settings, then click **Install**.

Step 2: Set Git Bash as Default in GitHub Desktop

1. Open your **GitHub Desktop** client.
2. In the top-left menu bar, click on **File** and select **Options** (on Mac, click *GitHub Desktop > Settings*).
3. Click on the **Integrations** tab on the left sidebar.
4. Look for the **Shell** dropdown menu.
5. Select **Git Bash** from the list of available terminal choices.
6. Click **Save** to apply the configuration.

Step 3: Open Git Bash Instantly

Once configured, you can launch Git Bash directly from your active project repository using either of these quick methods:

- **The Shortcut:** Press **Ctrl + ``** (Control + Backtick) on your keyboard.
- **The Menu:** Click on **Repository** in the top menu bar, then click **Open in Git Bash**.

Working with Git and Bash commands - Local Repo

Working with Git Bash commands

Getting Started Git Bash commands: <https://typoapp.io/blog/git-bash-commands>

These commands are used in Git bash terminal to navigate in the Git repo. So, practice these commands as these are very necessary.

Important Git Bash commands in snapshot.

File Management

Create a new directory

mkdir new-project

Create a new file

touch README.md

Copy files

cp original.txt copy.txt

cp -r source-folder/ destination-folder/ // Copy directory

Move or rename files

mv oldname.txt newname.txt

mv file.txt /path/to/destination/

Delete files and directories

rm unwanted.txt

rm -rf old-directory/ // Be careful with this!

Directory Navigation

Show current directory

pwd

List files and directories

ls

ls -la // Show hidden files and details

Change directory

cd project-folder

cd .. // Go up one level

Working with Git commands

Important Git local Repository Management Commands in snapshot.

Creating and Cloning Repositories

```
# Initialize a new repository or Setup Git in development (working) folder
git init

# Clone an existing repository
git clone https://github.com/username/repository.git

# Clone to a specific folder
git clone https://github.com/username/repository.git custom-folder-name
```

Tracking Changes

```
git status          //Check repository status
git add filename.txt // Add specific file
git add .           // Add files to staging area (all changes in folder)
git add *.js        // Add all JavaScript files
git add src/        // Add entire directory
git commit -m "Add user authentication feature" // Commit changes
git commit --amend -m "Updated message" // Amend the last commit
```

Viewing History

```
git log              // View commit history
```

Important Git Branch Management Commands in snapshot.

Branching is where Git's power truly shines, allowing parallel development streams.

Branch Management

```
git branch          // Local branches
git branch -r       // Remote branches
git branch -a       // All branches
git branch feature-login // Create a new branch
git checkout -b feature-payment # Create and switch to a new branch
git checkout main    // Switch branches
git branch -m old-name new-name // Rename a branch
git branch -d feature-complete // Delete a branch
git branch -D feature-broken    // Force delete
```

Syncing with Remotes

```
git push origin feature-branch // Upload local changes
git branch --set-upstream-to=origin/main main // Set up branch tracking
```

Git workflow

1. Navigate to the directory where you want to use Git

```
$ mkdir test , $cd test
```

2. Initialize new repository

When working with Git, people often mention Git repository, short form as repo. Earlier we installed Git, that doesn't mean we can use it. We need to manually tell machines when to use Git and in which directory. We need to create a Git repository in the directory where we need to use Git. So, Git starts saving history in the directory after Git repo is set up.

Use git init to create a new git repository. Before we can do anything git-related, we must initialize a repo first!. This is something you do once per project. Initialize the repo in the top-level folder containing your project. Git tracks the directory and **all nested directories**.

More information on Git init command can be found at www.git-scm.com/docs/git-init

```
$ git init
```

3. Create feature branch

```
$ git checkout -b feature-payment
```

4. Make changes in initialized folder and check status

Git is Watching Us after init command. Try making a new file in your repo and then run git status again. You'll see that git noticed the file, but it is "untracked. gives information on the current status of a git repository and its contents.

More information on Git init command can be found at www.git-scm.com/docs/git-status

```
$ git status
```

5. Stage the changes

```
$ git add . // add dot says that group all the changes done in current and nested project directory.
```

6. Create checkpoint

"Commit" command will add check point to the history saved in ". Git" folder with a message to refer.

```
$ git commit "your message for checkpoint reference."
```

7. Push changes to remote branch

```
$ git push <remote> <branch>
```

Create remote Repo

How do I get my code on Github

How do I get the local Git repo to GitHub. How do I connect them and How do I setup GitHub repo and what do I do on my machine to tell about GitHub repo. There are 2 different approaches to do this.

Option 1: Existing Repo

If you already have an existing repo locally that you want to get on GitHub.

- Create a new repo on GitHub.
- Connect your local repo (add a remote)
- Push up your changes to GitHub.

Option 2: Start From Scratch

If you haven't begun work on your local repo, you can...

- Create a brand-new repo on GitHub.
- Clone it down to your machine.
- Do some work locally.
- Push up your changes to GitHub.

Pushing: To get your own changes and Git history up on GitHub, we need to PUSH them up. The typical workflow looks like this:

- Make some changes locally.
- Add and commit to those changes.
- Repeat...
- Push new commits up to GitHub.

Option 1

Option 1 Move the local Git repo to empty GitHub branch. Perform <code>\$ git log</code> lot of commits get displayed. All these history and code/work will be moved to GitHub.	
Step 1: Create a new repo on GitHub	1. Sign in on GitHub. 2. In the upper-right corner of page next to profile picture, click “+ - > New Repository.” 3. In new page, enter the Repository name under “Create a new repository.” 4. Enter “Description” 5. Mark it as public or private. If it’s your website, mark it as private, otherwise anyone can make changes to the website. Now we have done empty repository on GitHub. Local repo doesn’t know anything about it.
Step 2: Give a name to GitHub url and set that name in the local Git repo.	Before we can push anything up to GitHub, we need to tell Git about our remote repository on GitHub. We need to set up a "destination" to push up to. In Git, we refer to these "destinations" as remotes. Each remote is simply a URL where a hosted repository lives. To view any existing remotes for you repository, we can run <code>git remote</code> or <code>git remote -v</code> (verbose, for more info) <u>\$ git remote -V</u> This just displays a list of remotes. Since we have not yet set the remote, this command will not show anything. Adding A New Remote: A remote is really two things: a URL and a label. To add a new remote, we need to provide both to Git. <u>\$ git remote add <name> <url></u> Ex: <code>\$ git remote add origin www.bhal.com</code> //Tells Git anytime name "origin" is used, it’s referring to this particular GitHub repo URL. Remember, by setting up a remote we are just telling Git about a remote repository URL. We have not "communicated" with the GitHub repo at all yet.
They are not commonly used, but there are commands to rename and delete remotes if needed. <code>git remote rename <old> <new></code> <code>git remote remove <name></code>	
Step 3: Push changes to remote GitHub	Now that we have a remote set up, let's push some work up to GitHub! To do this, we need to use the <code>git push</code> command. We need to specify the remote we want to push up to AND the specific local branch we want to push up to that remote. <u>\$ git push <remote> <branch></u> Ex: <u>\$ git push origin master</u> //tells git to push up the local-master branch to our origin remote. For the first time, there are no branches on remote. It creates a master branch. From next time, this command pushes local-branch to remote-master by default. But if we don’t want to push to remote-master <code>\$ git push origin <local-branch>: <remote-branch></code> <code>\$ git switch -c cats</code> <code>\$ git push origin cats:master</code> // This pushed cats to master branch

-u option allows us to set the upstream of the branch we are pushing. You can think of this as a link connecting our local branch to a branch on GitHub. Running **\$ git push -u origin master** sets the upstream of the local master branch so that it tracks the master branch on the origin repo. Local Git remembers it for future push commands.

Now that it remembers, do some changes and commit.

```
$ git switch -c dogs
```

```
$ git push -u origin dogs
```

```
$ git touch dogs.txt
```

```
$ git commit -m "text push"
```

```
$ git push // changes are pushed to existing dogs branch, not to master.
```

Step 4: Check GitHub repo	Check the recent commits with time reflecting on GitHub. Check "All commits" page.
---------------------------	--

Option 2 - Git clone

Connecting GitHub repo to local repo

So far, we've created our own Git repositories from scratch, but often we want to get a local copy of an existing repository instead. This opens the path for collaboration and exposure. Or you haven't started your work locally. You know you want to use GitHub. You can start creating empty repo on GitHub and clone it back to local machine. **This automatically configure remote.** Then do some work and push changes to GitHub repo.

To do this, we can clone a remote repository hosted on GitHub. All we need is a URL that we can tell Git to clone for use. To clone a repo, simply run `git clone <url>`. Git will retrieve all the files associated with the repository and will copy them to your local machine. In addition, Git initializes a new repository on your machine, giving you access to the full Git history of the cloned project. Make sure you are not inside of a repo when you clone!. For more information refer : www.git-scm.com/docs/clone

\$ git clone https://github.com/...

// Do some changes.

\$ git diff

\$ git push origin master

Anyone can clone a repository from GitHub, **provided the repo is public.** You do not need to be an owner or collaborator to clone the repo locally to your machine. You just need the URL from GitHub. You need permission to push up your own changes to the GitHub repo.

url for cloning can be copied from project path as shown below.

